

🌟 What is Jinja2 Templating Engine?

Jinja2 is the template engine used in Flask to create dynamic web pages.

📌 Meaning:

When we build websites, we don't want to write static HTML.

We want HTML that can change based on data from the server — for example:

- Show username dynamically
- Display list of items
- Use loops and conditions in HTML

To do this, Flask uses Jinja2, where we write HTML + special placeholders like `{{ }}` and `{% %}`.

🧠 Why We Use Jinja2?

Flask Sends Data Browser Sees

Python backend data Dynamic HTML page

Database values User-friendly webpage

Jinja2 helps us connect Python data → HTML pages.

🔥 Jinja2 Syntax

Purpose	Syntax	Example
Print variable	<code>{{ variable }}</code>	<code>{{ name }}</code>
Logic / Loop / Condition	<code>{% ... %}</code>	<code>{% for item in items %}</code>

✅ Simple Example

Python (app.py)

```
from flask import Flask, render_template
```

```
app = Flask(__name__)
```

```
@app.route('/')  
  
def home():  
    name = "Jani"  
  
    return render_template("index.html", username=name)
```

```
if __name__ == '__main__':
```

```
    app.run(debug=True)
```

HTML (templates/index.html)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Welcome</title>
```

```
</head>
```

```
<body>
```

```
    <h1>Hello, {{ username }} 🙌 </h1>
```

```
</body>
```

```
</html>
```

✅ Output:

Hello, Jani 🙌

Here,

- {{ username }} is Jinja2 placeholder
- Flask replaces it with "Jani" before sending to browser.

Example with Loop

app.py

```
@app.route('/students')
```

```
def students():
```

```
    names = ["Raju", "Ramya", "Jani"]
```

```
return render_template("students.html", students_list=names)
```

students.html

```
<h2>Students:</h2>
```

```
<ul>
```

```
{% for s in students_list %}
```

```
<li>{{ s }}</li>
```

```
{% endfor %}
```

```
</ul>
```

Result:

Students:

- Raju

- Ramya

- Jani

Key Benefits of Jinja2

Feature	Advantage
Template inheritance	Avoid repeating HTML
Loops & Conditions in HTML Dynamic display	
Variables & Filters	Format data easily
Clean separation	HTML (frontend) and Python (backend) separated

Conclusion

Jinja2 = HTML + Python variables + conditions + loops

It helps us display dynamic data on webpages using Flask.

What is Template Inheritance in Flask?

👉 Definition:

Template inheritance in Flask means **reusing a common layout (HTML structure)** across multiple web pages using the **Jinja2 templating engine**.

It allows you to define a **base HTML file (parent template)** that contains the common parts of your website — like the header, navbar, and footer — and then let other pages (**child templates**) fill in only the unique content for each page.

🧱 Why We Use Template Inheritance (Uses)

Purpose	Explanation
✅ Code Reusability	You don't repeat the same HTML (like header, footer, CSS links) in every page.
✅ Easy Maintenance	If you change something in the base template (e.g., navbar), it automatically updates in all pages.
✅ Consistency	All web pages have the same structure and design layout.
✅ Better Organization	Keeps your templates clean and modular — separating layout from page-specific content.

🔧 How It Works (Step-by-Step)

Step 1: Create a Base Template (base.html)

This is the **parent template**.

```
<!-- templates/base.html -->
```

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <title>{% block title %}My Website{% endblock %}</title>
```

```
</head>
```

```
<body>
```

```
<header>

  <h1>Welcome to My Flask Site</h1>

  <nav>

    <a href="/">Home</a> |

    <a href="/about">About</a> |

    <a href="/contact">Contact</a>

  </nav>

</header>


<hr>


<!-- This is where the child templates will insert their content -->

<div>

  {% block content %}

  {% endblock %}

</div>


<hr>

<footer>

  <p>&copy; 2025 My Website</p>

</footer>

</body>

</html>
```

Explanation:

- {% block title %} and {% block content %} → are **placeholders** that child templates can override.
 - Everything else (header, footer, etc.) is shared.
-

Step 2: Create Child Templates (like home.html, about.html)

These **extend** the base template and fill in the blocks.

Example 1: home.html

```
<!-- templates/home.html -->

{% extends "base.html" %}


{% block title %}Home Page{% endblock %}


{% block content %}

<h2>This is the Home Page</h2>

<p>Welcome to our Flask website!</p>

{% endblock %}
```

Example 2: about.html

```
<!-- templates/about.html -->

{% extends "base.html" %}


{% block title %}About Us{% endblock %}


{% block content %}

<h2>About Our Company</h2>

<p>We are a software training institute.</p>

{% endblock %}
```

Step 3: Create Flask Routes (app.py)

```
from flask import Flask, render_template

app = Flask(__name__)


@app.route('/')

```

```
def home():  
    return render_template('home.html')  
  
@app.route('/about')  
def about():  
    return render_template('about.html')  
  
if __name__ == '__main__':  
    app.run(debug=True)
```

Folder Structure

```
project/  
|  
├── app.py  
└── templates/  
    ├── base.html  
    ├── home.html  
    └── about.html
```

Output Behavior

- When you visit /, Flask loads **home.html**, which **extends** base.html → you get the base layout + home content.
 - When you visit /about, it loads **about.html**, again **reusing the same header, footer, etc.** from base.html.
-

In Short:

Term Meaning

extends Used in child template to inherit parent template (base.html).

Term Meaning

block A section in the base template that child templates can replace.

Real-Life Example Analogy:

Think of the **base template** as a **frame of a photo** — it's the same for all pictures.

Each **child template** is a **different photo** inserted inside the same frame.

So all pages look consistent, but their inner content is different.

Project Structure

flask_inheritance_demo/

|

├── app.py

└── templates/

 ├── base.html

 ├── home.html

 ├── about.html

 └── contact.html

Step 1: Base Template (templates/base.html)

This file holds the **common design** — navbar, footer, CSS, etc.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <title>{% block title %}My Flask Site{% endblock %}</title>
```

```
    <style>
```

```
        body {
```

```
font-family: Arial, sans-serif;

background-color: #f4f6f8;

margin: 0;

padding: 0;
}

header {

background-color: #007bff;

color: white;

padding: 15px;

text-align: center;
}

nav a {

color: white;

margin: 0 15px;

text-decoration: none;

font-weight: bold;
}

nav a:hover {

text-decoration: underline;
}

.container {

margin: 30px;

background: white;

padding: 20px;

border-radius: 10px;

box-shadow: 0px 0px 10px rgba(0,0,0,0.1);
}

footer {
```

```
        background-color: #007bff;
        color: white;
        text-align: center;
        padding: 10px;
        position: fixed;
        bottom: 0;
        width: 100%;
    }
</style>
</head>
<body>
    <header>
        <h1>My Flask Website</h1>
        <nav>
            <a href="/">Home</a>
            <a href="/about">About</a>
            <a href="/contact">Contact</a>
        </nav>
    </header>

    <div class="container">
        {% block content %}
        <!-- Child templates will insert their content here -->
        {% endblock %}
    </div>

    <footer>
        <p>&copy; 2025 My Flask Website</p>
```

```
</footer>

</body>

</html>
```

Step 2: Home Page (templates/home.html)

```
{% extends "base.html" %}
```

```
{% block title %}Home - My Flask Site{% endblock %}
```

```
{% block content %}
```

```
<h2>Welcome to My Flask Site</h2>
```

```
<p>This is the home page. We use template inheritance to reuse layout and styles!</p>
```

```
{% endblock %}
```

Step 3: About Page (templates/about.html)

```
{% extends "base.html" %}
```

```
{% block title %}About - My Flask Site{% endblock %}
```

```
{% block content %}
```

```
<h2>About Us</h2>
```

```
<p>We are passionate developers teaching Flask web development with real examples.</p>
```

```
{% endblock %}
```

Step 4: Contact Page (templates/contact.html)

```
{% extends "base.html" %}
```

```
{% block title %}Contact - My Flask Site{% endblock %}
```

```
{% block content %}
<h2>Contact Us</h2>
<p>Email: support@flaskdemo.com</p>
<p>Phone: +91 9876543210</p>
{% endblock %}
```

Step 5: Flask App (app.py)

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def home():
    return render_template('home.html')

@app.route('/about')
def about():
    return render_template('about.html')

@app.route('/contact')
def contact():
    return render_template('contact.html')

if __name__ == '__main__':
    app.run(debug=True)
```

Step 6: Run the Application

In your terminal (inside the project folder):

```
python app.py
```

Then open your browser and visit:

-  <http://127.0.0.1:5000/> → Home Page
 -  <http://127.0.0.1:5000/about> → About Page
 -  <http://127.0.0.1:5000/contact> → Contact Page
-

Output Summary

All three pages:

- Share the same **header**, **navbar**, and **footer**.
 - Only the **content** area changes.
 - Any change in base.html (like navbar color or title) will reflect on every page instantly.
-

Simple Real-Life Comparison

Concept	Real-life Example
base.html	A reusable PowerPoint slide layout
home.html, about.html, contact.html	Individual slides using that layout
Change once in layout → all slides update automatically 	

render_template() is used to connect flask app and html ?

Yes  — **that understanding is correct**, but let me explain it **clearly and simply**, the way freshers should remember it.

Short Answer

 **render_template() is used to connect the Flask (Python) backend with HTML templates.**

What does `render_template()` actually do?

`render_template()` does **two main jobs**:

1 It loads an HTML file

From the **templates/** folder

2 It sends data from Flask (Python) to HTML

So HTML can display dynamic content using **Jinja2**

Simple Definition (Exam / Interview Friendly)

`render_template()` is a Flask function used to render (display) an HTML page and pass data from the Flask application to the HTML template.

Basic Syntax

`render_template("file_name.html", variable=value)`

Simple Example

♦ Flask (Python)

```
from flask import Flask, render_template
```

```
app = Flask(__name__)
```

```
@app.route("/")
```

```
def home():
```

```
    name = "Jani"
```

```
    return render_template("home.html", username=name)
```

```
app.run(debug=True)
```

♦ HTML (home.html)

```
<h1>Welcome {{ username }}</h1>
```

♦ Output in Browser

Welcome Jani

How Flask & HTML are Connected

Flask (Python) **HTML (Jinja2)**

`render_template()` Receives data

Variables `{{ variable }}`

Logic `{% if %}`, `{% for %}`

Real-Time Analogy

Think like this 

- **Flask** → Brain  (logic, data)
 - **HTML** → Face  (display)
 - **`render_template()`** → Messenger 
(takes data from brain and shows it on the face)
-

Final One-Line Answer

 **Yes, `render_template()` connects Flask backend logic with HTML frontend templates and allows dynamic data display.**

If you want, I can also explain:

- Difference between `render_template()` and `redirect()`
- What happens internally when `render_template()` is called
- Interview questions on `render_template()`